BIOT 6900 · AI-POWERED BIOTECH WORKFLOWS

Module 3 Lab Guide: Protein Language Models for Target Triage & Variant Effects

Module 3 lab · Part 1 (toy model) and Part 2 (real data) · Fall 2026

What this lab is

This lab teaches the method you will use in the Module 3 assignment: using protein language models to ask **does a protein resemble proteins we already know how to drug, and which variants does the model find surprising?** You practise it twice, first on a toy model you can see inside, then on real human proteins.

The lab has **two parts** that repeat the same steps at increasing realism. Neither is graded; the graded work is the separate **Module 3 assignment**, which applies the method to your own targets.

Part	Data	Setting	Graded?
Part 1 · Toy model	Synthetic families you can see inside	In class, Week 3	No (practice)
Part 2 · Real data	About 1,200 real human proteins (data pack)	Self-paced, on your own	No (practice)
Module 3 assignment	Your own Module 2 targets, any disease	Independent (separate sheet)	Yes · Lab 3

The one idea to carry through

A protein language model gives you a **prior, not a verdict**. It tells you which proteins look like past drug targets and which changes evolution disfavours. Both parts show what the prior can tell you, and where it stops.

Maps to course outcomes: LO1 (apply industry AI tools), LO3 (interpret outputs in biological context), LO4 (evaluate model limitations and validation), LO5 (communicate through professional documentation).

The big idea, in five beats

- **Hide a residue, predict it.** Models learn by predicting masked amino acids, which forces them to learn conservation and protein families.
- **Embeddings are coordinates.** Each protein becomes a list of numbers; similar biology gives nearby numbers.
- **A prior comes from labels.** The model knows nothing about drugs. A classifier trained on known drug targets turns embeddings into a tractability estimate.
- **Validate by family.** Random splits leak close relatives into training and inflate performance. Hold out whole families for the honest number.
- **Variant scores are zero-shot.** `log P(mutant) − log P(wild-type)` needs no labels, but it sees missense changes only and is not a clinical verdict.

Before you start

1 · Your environment

Use the `biot6900` conda environment. **Part 1** needs only numpy and pandas (plots appear if matplotlib is installed). **Part 2** also needs scikit-learn and matplotlib. Install them once:

```
conda activate biot6900
conda install scikit-learn matplotlib
```

No model downloads are needed. All embeddings and variant scores for Part 2 are pre-computed in the Module 3 data pack.

2 · The files you need

File	Needed for	Where to get it
`BIOT6900_Module3_Part1_ToyModel.ipynb`	Part 1	Canvas
`BIOT6900_Module3_Part2_RealData.ipynb`	Part 2	Canvas, with the data pack
Module 3 data pack (zip)	Part 2 (and the assignment)	Canvas; unzip into `data/`

Your folder should look like this:

```
your-repo/
├── BIOT6900_Module3_Part1_ToyModel.ipynb
├── BIOT6900_Module3_Part2_RealData.ipynb
└── data/
    ├── panel.tsv                panel_emb_main.npy      panel_emb_baseline.npy
    ├── proteome.tsv             proteome_emb_main.npy   proteome_emb_baseline.npy
    ├── variant_scores_TP53.tsv  variant_scores_APOE.tsv variant_scores_TREM2.tsv
    └── anchor_variants.tsv      manifest.json
```

Part 1 · Toy model (in class, not graded)

Everything in Part 1 is **synthetic**: twelve invented protein families, so you know the right answer and can check the method against it. It runs offline in seconds. Work in pairs; one person drives, one writes the answers.

Step 1.1 · Run the setup cell

Open the notebook and run the first code cell (Shift+Enter).

Observe: a line starting `Ready.` with your numpy and pandas versions, and `plots: True` (or `False` if matplotlib is missing; tables still print).

Step 1.2 · Build the toy protein world (Section 1)

Run the cell that creates the families. Six families are druggable: five share a pocket motif `GSGKST` at positions 15–20, and **Channel-like** is druggable through a different pocket that no other family has.

Observe: a table of 12 families with 15 members each, and five Kinase-like sequences printed with the pocket motif separated out.

✎ **Your turn:** which positions are identical in all five sequences, and why would evolution keep them fixed?

Step 1.3 · Hide a residue and predict it (Section 2)

The toy model reads the rest of the sequence to work out the family, then predicts the hidden residue from that family's amino-acid frequencies at that position. Run the first prediction cell (a **conserved** position), then the second (a **variable** position).

> **What you should see**
>
> **Conserved position:** the true residue gets about **0.62** probability.
>
> **Variable position:** the model prefers the family's usual residue (about **0.38**) and gives the true residue only about **0.10**.
>
> That difference is the conservation signal from lecture slide 6.

✎ **Your turn:** change `vpos` to other variable positions and explain why the model is confident in one case and unsure in the other.

Step 1.4 · Embeddings and clusters (Section 3)

Each toy protein becomes **64 numbers** (real ESM C uses 960). The embedding is never told the family names.

Observe: an embedding matrix of shape (180, 64); nearest neighbour in the same family for about **94%** of proteins versus **8%** by chance; and a 2D plot where families form clusters.

✎ **Your turn:** why do the families cluster if the embedding never saw their names?

Step 1.5 · A toy druggability prior (Section 4)

A logistic regression predicts "druggable" from the embeddings, tested two ways: a **random split** (relatives stay in training) and a **family split** (whole families held out).

> **What you should see**
>
> Accuracy about **96%** with a random split, about **78%** with a family split. The gap is homology leakage.
>
> In the family split, **Channel-like scores about 0.40** even though it is druggable: its pocket appears nowhere in training. Labels encode history.

Structural-like scores about 0.55, a false positive.

✏️ **Your turn:** answer the two questions (and the bonus) under Section 4 in 2–3 sentences each.

Step 1.6 · Variant scoring (Section 5)

The score is `log P(mutant) − log P(wild-type)`. Strongly negative means the model is surprised.

Observe: the same substitution to tryptophan scores about **−3.4** at a conserved or pocket position and about **−1.6** at a variable position.

✏️ **Your turn:** predict each score before running the last code cell, then change `pos` and `mut` to test your own variant. Note that `pos` is 0-based in this cell (5 means residue 6).

Step 1.7 · Wrap-up questions

Answer the three questions at the end of the notebook in one or two sentences each. **Keep your answers:** they are a good starting point for your Module 3 assignment report.

Part 2 · Real data (self-paced, not graded)

Part 2 repeats the Part 1 steps on **real human proteins**, using pre-computed ESM C embeddings and real Open Targets tractability labels. Work through it on your own with this guide; your exact numbers depend on the data pack, so this guide describes **what to look for** rather than fixed values.

Step 2.1 · Load the data pack

```
panel = pd.read_csv("data/panel.tsv", sep="\t")
X = np.load("data/panel_emb_main.npy").astype("float32")  # float16 on disk
print(panel.shape, X.shape)
print(json.load(open("data/manifest.json"))["embedding_models"])
```

Observe: about 1,200 panel proteins and one embedding row per protein (960 numbers for ESM C 300M). The manifest tells you exactly which models produced the embeddings.

Step 2.2 · Understand the labels

Each protein has Open Targets **clinical precedence** flags: `sm_clinical` (small molecule), `ab_clinical` (antibody), and `any_clinical`. Labels come from Approved, Advanced Clinical, or Phase 1 evidence only.

Two things to notice

The panel is **enriched**: about one third of proteins have clinical precedence, far more than across the whole proteome. Read predicted probabilities as **rankings**, not absolute chances.

The "Druggable Family" label was deliberately left out. It is assigned from protein family, so using it would build the answer into the label.

Step 2.3 · Map the embedding space

Project the embeddings to 2D (PCA), colour the eight largest family groups, and mark **TP53**.

Observe: families such as kinases and GPCRs form visible clusters, as in the toy. Note which proteins sit near p53.

Step 2.4 · Nearest neighbours

Using cosine similarity, list the five nearest panel proteins to TP53 and to a kinase of your choice.

Hint. normalise each row to length 1, then `Z @ Z.T` gives all cosine similarities at once.

Observe: a kinase's neighbours should be mostly other kinases. For p53, note what kinds of proteins its neighbours are, and whether any have clinical precedence.

Step 2.5 · Train and validate the tractability prior

Fit a scaled logistic regression on `any_clinical`, and evaluate it with **5-fold random CV** (`StratifiedKFold`) and **5-fold family CV** (`GroupKFold` with `groups=panel.family_group`). Report two metrics:

- **AUROC:** how well the model ranks positives above negatives (0.5 = chance).
- **Average precision (AP):** precision–recall summary. Its chance level equals the positive fraction (about 0.33 here), which is why it is more honest than accuracy under class imbalance.

Observe: the family-split scores are lower than the random-split scores. The size of that gap on real proteins is the key result of Part 2.

Step 2.6 · Where does p53 land?

Look up p53's **out-of-fold** probability from the family split (a prediction made without p53 in training).

Discuss: p53 has been one of the most sought-after and hardest cancer targets. Does the prior call it tractable? Either answer is a teaching point: sequence resemblance is not the same as a druggable pocket, and mutant-specific pockets (Y220C) are invisible to a model trained on historical labels.

Step 2.7 · ESM C versus ESM-2

Repeat Step 2.5 with `panel_emb_baseline.npy` (ESM-2 35M, 480 numbers). Compare the family-split AUROC and the Spearman correlation of the two models' predictions.

Observe: whether a model with about ten times more parameters gives a meaningfully better prior. If both agree, the ranking is more robust.

Step 2.8 · Variant scores for p53

```
v = pd.read_csv("data/variant_scores_TP53.tsv", sep="\t")   # all substitutions
anchors = pd.read_csv("data/anchor_variants.tsv", sep="\t")
anchors[anchors.gene == "TP53"]
```

Plot the distribution of all p53 substitution scores and mark the hotspots (R175H, Y220C, G245S, R248Q, R249S, R273H, R282W). The `percentile_in_protein` column tells you how extreme each hotspot is relative to every possible change in p53.

Observe: check whether the hotspots fall in the damaging (strongly negative) tail, and whether any sit closer to the middle. Ask why a cancer driver might not be the change evolution disfavours most.

Step 2.9 · Check-in (not graded)

1. How large was the random versus family gap on real proteins, and which number would you report to a discovery team?
2. Did ESM C clearly beat ESM-2? What does that suggest about model size for this task?
3. Name one reason a p53 hotspot might score as only moderately surprising.

Troubleshooting

Symptom	Likely cause & fix
FileNotFoundError: data/panel.tsv	The data pack isn't unzipped into `data/` next to the notebook. Check the folder name and that files aren't in a nested subfolder.
Part 1 prints plots: False	matplotlib isn't installed. Part 1 still works (tables print); install it with the command below for Part 2.
ModuleNotFoundError: sklearn	Run `conda install scikit-learn matplotlib` inside `biot6900`, then restart the kernel.
Embedding shape doesn't match the table	You loaded the wrong file or filtered one without the other. Rows in `panel_emb_*.npy` align with `panel.tsv`. Filter both with the same index.
Warnings about float16	Embeddings are stored as float16. Convert with `.astype("float32")` right after loading.
GroupKFold error about groups	The groups array must have one entry per row you train on. If you filtered the panel, pass the filtered `family_group` column.
Probabilities all near 0 or 1	Features weren't scaled. Use a pipeline with `StandardScaler` before `LogisticRegression`.

Symptom	Likely cause & fix
Notebook won't run top to bottom	Use Kernel → Restart & Run All and fix the first error before moving on.

What comes next

The **Module 3 assignment** (separate sheet on Canvas) applies exactly these steps to **your own targets from Module 2**, whichever disease you studied: a tractability prior with family-aware validation, small-molecule and antibody priors, nearest known drug targets, anchor variant scores, and a re-ranked target list handed to Week 4.

Keep your Part 1 and Part 2 notebooks

The assignment reuses the Part 2 code for loading the data pack, training the prior, and validating by family. Your Part 1 wrap-up answers are a good starting point for the assignment report.